

Proiectul 3

Programare Orientată pe Obiecte

Grupele 141 și 144

Obiectiv

Integrarea conceptelor de **excepții**, **template-uri**, **STL**, **std::string** și **Design Patterns** într-o aplicație C++ completă, conform materiei studiate în Laboratoarele 6, 7 și 8.

Studentii au la dispoziție **două variante**, dintre care trebuie aleasă una:

Varianta A — Extinderea Proiectului 2

Aplicația realizată în Proiectul 2 se extinde cu toate conceptele din Laboratoarele 6–8. Toate cerințele obligatorii ale Proiectelor 1 și 2 (încapsulare, constructori, destructor virtual, moștenire, clase abstracte, polimorfism, **dynamic_cast** etc.) rămân valabile și trebuie să fie prezente în codul predat. Colecțiile gestionate cu tablouri alocate dinamic (**new[]**) se vor înlocui cu containere STL.

Varianta B — Aplicație nouă de la zero

Se realizează o aplicație complet nouă, pentru un domeniu ales de student (ex.: bibliotecă, spital, magazin online, sistem de rezervări, joc etc.). Aplicația trebuie să respecte **toate** cerințele obligatorii din Proiectele 1 și 2 (încapsulare, ierarhie de moștenire, clasă abstractă, polimorfism, destructor virtual, **dynamic_cast**) **plus** toate cerințele obligatorii ale Proiectului 3 de mai jos. Domeniul ales trebuie menționat în **README.txt** împreună cu o scurtă descriere a ierarhiei de clase.

Indiferent de variantă, toate cerințele obligatorii de mai jos trebuie satisfăcute integral.

Cerințe obligatorii

1. Excepții custom și gestionarea erorilor prin excepții

Aplicația (extinsă sau nouă) trebuie să renunțe complet la codurile de retur pentru semnalarea erorilor și să utilizeze **exclusiv mecanismul throw/try/catch**.

Cerințe:

- Implementați o **ierarhie proprie de excepții** cu minimum 3 clase, moștenind din **std::exception** sau din subclasele sale standard (**std::runtime_error**, **std::logic_error** etc.). Fiecare excepție trebuie să suprascrie **what()** cu **noexcept override** și să conțină câmpuri specifice domeniului (ex.: valoarea invalidă, id-ul afectat, limita depășită).
- Toate metodele care pot eșua (constructori, setteri, operații CRUD) trebuie să arunce excepții adecvate în loc să returneze coduri de eroare sau să afișeze mesaje direct.

- Demonstrați **stack unwinding**: creați cel puțin un obiect cu resurse (mesaj în destructor) în interiorul unui bloc `try` și verificați că destructorul este apelat corect la propagarea excepției.
- Destructorii tuturor claselor trebuie marcați **noexcept** (implicit sau explicit) și nu trebuie să arunce nicio excepție.
- Utilizați **multiple blocuri catch**: prindeți mai întâi excepțiile derivate, apoi cele din bază, iar ca ultim resort `catch(...)`.

2. Funcții și clase template

Aplicația trebuie să conțină **minimum o clasă template** și **minimum două funcții template**, utilizate în mod real în logica aplicației (nu doar ca demonstrații izolate).

Cerințe:

- **Clasă template generică** (ex.: `Depozitar<T>`, `Colectie<T, N>`, `Rezultat<T>`) parametrizată cu cel puțin un tip. Clasa trebuie să gestioneze resurse (alocare/dealocare) și să integreze excepții (`overflow_error`, `underflow_error` etc.) la operațiile sale.
- **Minimum o metodă template** în interiorul clasei template (parametru de tip independent de cel al clasei, ex.: metodă de conversie sau copiere între tipuri diferite).
- **Minimum două funcții template** libere cu deducere automată de tip, folosite practic în aplicație (ex.: funcție de sortare generică, funcție de căutare generică, funcție de transformare).
- **Cel puțin o specializare totală** a unei funcții sau clase template pentru un tip specific, cu justificare în comentarii.
- Demonstrați utilizarea unui **parametru non-tip** (ex.: `template<typename T, int N>`) pentru a fixa dimensiunea unei colecții la compilare.

3. Containere și algoritmi STL

Toate colecțiile de obiecte din aplicație trebuie migrate de la tablouri alocate dinamic la **containere STL adecvate**, alese și justificate pentru fiecare caz de utilizare.

Cerințe:

- Utilizați **minimum 3 tipuri diferite de containere STL** (`vector`, `list`, `map`/`unordered_map`, `set`, `stack`, `queue`, `deque` etc.), fiecare ales în funcție de complexitatea operațiilor necesare. Justificați alegerea fiecărui container în comentarii (O-notație).
- Utilizați **minimum 5 algoritmi diferiți** din `<algorithm>` cu **lambda-uri**: `find_if`, `sort`, `count_if`, `remove_if`/`erase`, `for_each`, `transform`, `any_of`/`all_of`/`none_of`, `copy_if` etc.
- Implementați **CRUD complet** (*Create*, *Read*, *Update*, *Delete*) folosind containere STL și algoritmi, cu excepții pentru erorile posibile (element negasit, duplicat, capacitate depășită etc.).
- Demonstrați idioma **erase-remove_if** pentru `vector` și `remove_if` membră pentru `list`.

- Utilizați `std::pair` sau **structured bindings** (`auto [k, v]`) acolo unde returnați sau iterați perechi de valori.

4. Migrarea la `std::string`

Toate câmpurile de tip `char*` sau `char[]` din clasele proiectului trebuie **migrate la** `std::string`.

Cerințe:

- Înlocuiți toate câmpurile `char*/char[]` cu `std::string` și eliminați codul manual de gestionare a memoriei aferent (ex.: `new char[]`, `strcpy`, `delete[]`).
- Demonstrați că migrarea la `std::string` elimină necesitatea scrierii manuale a constructorului de copiere, `operator=` și destructorului pentru siruri (Rule of Three simplificat).
- Utilizați metodele `std::string` în logică reală: `find`, `substr`, `replace`, `to_string`, `stoi/stod`, `empty`, `size`.
- Utilizați `std::ostringstream` sau operatorul `+` pentru construirea mesajelor de excepție și a metodelor `toString()`.

5. Interfața `IObject` și identificatori unici

Clasele din ierarhia principală trebuie să implementeze o **interfață** `IObject` similară celei din Laboratorul 7.

Cerințe:

- `IObject` trebuie să conțină: contor static pentru generarea de id-uri unice, `getId() const noexcept`, metodă pur virtuală `toString() const`, `operator<<` (friend) apelat polimorfic și `operator==` pe bază de id.
- Toate clasele concrete din ierarhia aplicației trebuie să moștenească (direct sau indirect) din `IObject` și să implementeze `toString()` folosind `std::string`.

6. Design Pattern: Singleton

Aplicația trebuie să conțină **cel puțin un Singleton** implementat în varianta **Meyers** (variabila locală `static` în `getInstance()`).

Cerințe:

- Singleton-ul trebuie să joace un rol real în aplicație (ex.: `Logger`, `Configuratie`, `Registry` de obiecte, `NotificareManager`).
- Constructorul, constructorul de copiere și `operator=` trebuie dezactivate explicit cu `= delete`.
- Demonstrați în `main` că nu se pot crea două instanțe (prin comentariu sau eroare de compilare comentată).
- Singleton-ul trebuie să fie integrat cu restul aplicației: min. 3 clase sau operații să-l apeleze automat (ex.: constructorii claselor de domeniu loghează prin Singleton).

7. Design Pattern: Abstract Factory

Aplicația trebuie să conțină un **Abstract Factory** cu minimum **două familii de produse concrete** și **două tipuri de produse per familie**.

Cerințe:

- Definiți **minimum 2 interfețe de produs abstracte** și câte **2 implementări concrete** per interfață (câte una per familie).
- Definiți **fabrica abstractă** cu metodele factory corespunzătoare și **minimum 2 fabrici concrete**.
- **Clientul** (clasa care folosește fabrica) trebuie să primească fabrica prin **dependency injection** (constructor sau parametru de funcție) și să lucreze exclusiv cu interfețele abstracte, fără a cunoaște tipurile concrete.
- Demonstrați în `main` că **schimbarea familiei** se face printr-o singură modificare (înlocuirea fabricii).

8. Extinderea meniului interactiv

Meniul CRUD din Proiectele 1 și 2 trebuie extins pentru a acoperi funcționalitățile noi:

- Opțiuni de **filtrare și sortare** a colecției principale (cu algoritmi STL și lambda-uri).
- Opțiune de **selectare a canalului/familiei** la runtime (demonstrând Abstract Factory).
- Toate erorile din meniu tratate prin `try/catch`, cu mesaje clare și revenire la meniu (fără crash).
- Opțiune de **afișare a jurnalului** (demonstrând Singleton-ul Logger).

Cerințe suplimentare (bonus)

- Implementarea clasei template `Rezultat<T>` (similar cu `std::expected` din C++23): încapsulează fie o valoare, fie un mesaj de eroare, cu metode `ok()`, `get()`, `getEroare()` și o metodă template `transforma(F)`.
- Implementarea unui al treilea Design Pattern creational sau structural (ex.: **Builder**, **Prototype**, **Decorator**), cu justificarea alegerii în `README.txt`.
- Utilizarea `std::initializer_list<T>` pentru inițializarea colecțiilor cu sintaxă cu acolade.
- Utilizarea **parametrilor non-tip** în clasa template pentru a crea structuri de date cu dimensiune fixă pe stivă (fără `new`).
- Interfață grafică (ex.: ImGui) sau conectare la bază de date.

Condiții eliminatorii

NECOMPILAREA PROIECTULUI DUCHE LA NOTAREA PROIECTULUI CU NOTA 1.

LIPSA IERARHIEI DE EXCEPȚII CUSTOM SAU UTILIZAREA CODURILOR DE RETUR ÎN LOC DE EXCEPȚII DUCHE LA NOTAREA PROIECTULUI CU NOTA 1.

LIPSA CLASEI TEMPLATE SAU A CONTAINERELOR STL DUCHE LA NOTAREA PROIECTULUI CU NOTA 1.

Proiectul va fi verificat printr-un flux automatizat și prin cross-evaluation pe sursele predate.

Dacă în timpul rulării programul:

- Produce Undefined Behaviour (ex.: excepție aruncată dintr-un destructor în timpul stack unwinding, accesarea unui container gol fără verificare),
- Are funcționalități greșit implementate,
- Are funcționalități care eșuează la execuție,

se poate acorda punctaj parțial sau penalizare majoră.

Grilă de evaluare orientativă

Criteriu	Puncte
Ierarhie de excepții custom (min. 3 clase, <code>what()</code> override, câmpuri specifice domeniului)	15
Funcții și clase template (min. 1 clasă, 2 funcții, 1 specializare, parametru non-tip)	15
Containere STL (min. 3 tipuri, justificate) + algoritmi (min. 5 diferiți cu lambda-uri)	15
Migrare la <code>std::string</code> + eliminarea Rule of Three manual	10
Interfață <code>IObject</code> cu <code>toString()</code> , <code>getId()</code> , <code>operator<<</code>	5
Design Pattern Singleton Meyers integrat real în aplicație	10
Design Pattern Abstract Factory (2 familii, 2 produse/familie, dependency injection)	15
Mentținerea cerințelor din Proiectele 1 și 2 (Varianta A) sau implementarea lor completă de la zero (Varianta B)	5
Extinderea meniului CRUD (filtrare/sortare, selectare familie, jurnal)	10
Total	100
Bonus (Rezultat<T>, al 3-lea pattern, <code>initializer_list</code> , GUI/BD)	+10

Note tehnice

- **Compiler:** `g++ -std=c++17 -Wall -Wextra`
- Proiectul se predă ca arhivă `.zip` conținând toate fișierele sursă (`.cpp`, `.h`) și un fișier `README.txt` cu: varianta aleasă (A sau B), instrucțiuni de compilare, o scurtă descriere a aplicației și o justificare a alegerii containerelor STL utilizate. Pentru Varianta B, `README.txt` trebuie să descrie și ierarhia de clase și domeniul ales.
- Spre deosebire de Proiectele 1 și 2, **utilizarea containerelor STL este obligatorie** în Proiectul 3. Pentru Varianta A, tablourile alocate dinamic (`new[]`) pentru colecții de obiecte trebuie înlocuite cu containere STL. Pentru Varianta B, colecțiile se implementează direct cu STL de la început.
- Implementarea template-urilor trebuie să se afle **integral în fișierele .h** (nu în `.cpp` separat), deoarece compilatorul generează codul concret la instantiere.
- Fișierele sursă trebuie organizate logic: câte un fișier `.h` și un fișier `.cpp` per clasă (sau grupate logic), plus un `main.cpp`. Clasele și funcțiile template se pot grupa într-un singur header.

Tabel sintetic al conceptelor acoperite

Concept	Laborator sursă	Cerință asociată
Excepții custom + ierarhie	Lab 6, §2.8	Cerința 1
Stack unwinding + <code>noexcept</code>	Lab 6, §2.5–2.7	Cerința 1
Funcții template (1–2 tipuri)	Lab 6, §3.2–3.3	Cerința 2
Specializare totală	Lab 6, §3.4	Cerința 2
Clase template + metode template	Lab 6, §4–5	Cerința 2
Parametri non-tip	Lab 6, §4.3	Cerința 2
Iteratori + lambda-uri	Lab 7, §1	Cerința 3
Containere STL	Lab 7, §2	Cerința 3
Algoritmi <code><algorithm></code>	Lab 7, §3	Cerința 3
<code>IObject</code> + CRUD cu STL	Lab 7, §4–5	Cerințele 3, 5
<code>std::string</code>	Lab 8, §1	Cerința 4
Listă de inițializare a membrilor	Lab 8, §2.1	Cerințele 1, 2
Singleton (Meyers)	Lab 8, §4	Cerința 6
Abstract Factory	Lab 8, §5	Cerința 7

Planificare orientativă (10 zile)

Ziua	Activitate	Detalii
1	Planificare și arhitectură	Alegeți domeniul (extensie Proiect 2 sau nou). Schițați ierarhia de clase, excepțiile, containerele și pattern-urile. Creați structura de fișiere.
2	Ierarhie de excepții	Implementați cele min. 3 clase de excepții cu câmpuri specifice și <code>what()</code> . Integrați în constructori și setteri.
3	Migrare <code>std::string</code> + <code>IObject</code>	Înlocuiți toate câmpurile <code>char*/char[]</code> cu <code>std::string</code> . Implementați <code>IObject</code> . Actualizați <code>toString()</code> pentru fiecare clasă.
4	Clase și funcții template	Implementați clasa template generică, metodele template, funcțiile template libere și specializarea totală.
5	Containere STL	Migrați colecțiile la containere STL potrivite. Documentați alegerea în comentarii (complexitate).
6	Algoritmi și CRUD complet	Implementați <code>find_if</code> , <code>sort</code> , <code>count_if</code> , <code>remove_if</code> /erase, <code>transform</code> etc. CRUD cu excepții la fiecare operație.
7	Singleton Logger	Implementați Meyers Singleton. Integrați în constructori, destructori și setteri. Adaugați opțiune de jurnal în meniu.
8	Abstract Factory	Implementați interfețele și produsele concrete, cele două fabrici și clientul cu dependency injection. Integrați în meniu.
9	Extinderea meniului + integrare	Adaugați opțiunile de filtrare, sortare și selectare familie. Verificați coerența întregii aplicații.
10	Testare, bonus și documentare	Testați toate scenariile de eroare. Implementați bonus dacă este timp. Scrieți <code>README.txt</code> . Arhivați.